NoSQL Databases

Java Backend Interview — Short Notes

1. SQL vs NoSQL — Key Differences

Dimension	SQL (Relational)	NoSQL
Schema	Fixed schema — defined upfront. ALTER TABLE to change.	Flexible/dynamic schema — different documents can have different fields.
Data model	Tables with rows and columns. Normalized relationships.	Documents, Key-Value, Wide-Column, Graph — model varies by type.
Joins	Native JOIN support — relational model strength.	Typically no joins. Denormalize or embed related data.
ACID	Full ACID by default.	Varies: MongoDB multi-doc ACID (4.0+), Cassandra eventually consistent, DynamoDB per-item ACID.
Scaling	Vertical scaling primary. Horizontal sharding complex.	Designed for horizontal scale-out from the ground up.
Query language	SQL — standardized, powerful.	DB-specific APIs. MongoDB MQL, Cassandra CQL, DynamoDB PartiQL.
Consistency	Strong consistency by default.	Often eventual consistency. Tunable in Cassandra/DynamoDB.
Use when	Complex queries, transactions, structured data, financial systems.	High scale, flexible schema, specific access patterns, unstructured data.

2. NoSQL Types & Realistic Use Cases

Type	Examples	Data Model	Strengths	Realistic Use Cases
Document	MongoDB, Firestore, CouchDB	JSON/BSON documents. Nested fields. Arrays.	Flexible schema, rich queries, embedded data, indexing.	Product catalog, user profiles, CMS, e-commerce orders with variable attributes.
Key-Value	Redis, Dynamo DB, Riak	Hash map: key → value (any type).	O(1) lookups, massive throughput, simple data.	Sessions, caching, shopping cart, distributed locks, counters, leaderboards.
Wide-Column	Apache Cassandra, HBase, BigTable	Rows with dynamic columns. Column families. Sorted by row key.	Linear horizontal scale, high write throughput, time-series.	IoT sensor data, activity feeds, metrics, audit logs, user event history.
Graph	Neo4j, Amazon Neptune, ArangoDB	Nodes (entities) + Edges (relationships) + Properties.	Efficient relationship traversal. Variable-depth queries.	Social networks (friends-of-friends), fraud detection, recommendation engines, knowledge graphs.
Search	Elasticsearch, OpenSearch, Solr	Inverted index. Documents with analyzed fields.	Full-text search, faceting, relevance scoring, aggregations.	Product search, log analysis, autocomplete, monitoring dashboards.
Time-Series	InfluxDB, TimescaleDB, Prometheus	Measurements tagged with timestamp.	Optimized time-range queries, downsampling, rollups.	Application metrics, server monitoring, financial tick data, IoT sensor readings.

3. MongoDB Deep Dive

3.1 Data Modeling — Embed vs Reference

Approach	When to Use	Example	Pros / Cons
Embed (denormalize)	Data always accessed together. 1:1 or 1:few. Child doesn't grow unboundedly.	User document embeds addresses array. Order embeds order items.	+ Single read. + Atomic update. - Document size limit 16MB. - Duplication.
Reference (normalize)	Data accessed independently. 1:many with large sets. Shared across documents.	Order references product_id. Comment references user_id.	+ No duplication. + Independent updates. - Requires $lookup (join). - Multiple reads.

```
// Embed: user with embedded addresses (always fetched together)
{ _id: ObjectId, name: 'Alice',
addresses: [{type:'home',city:'NY'}, {type:'work',city:'SF'}] }
// Reference: order references product
{ _id: ObjectId, order_date: ISODate, product_id: ObjectId('...') }
// $lookup (left outer join)
db.orders.aggregate([
{ $lookup: { from:'products', localField:'product_id',
foreignField:'_id', as:'product' } }
])
```

3.2 MongoDB Indexing & Transactions

• **Indexes**: Single field, Compound, Multikey (arrays), Text, Geospatial, Wildcard, TTL (auto-expire documents).

• **TTL Index**: automatically delete documents after N seconds — ideal for sessions, OTP, temporary data.

• **Transactions (4.0+)**: multi-document ACID transactions across collections and shards (4.2+).

```
// TTL index — expire after 3600 seconds
db.sessions.createIndex({createdAt:1}, {expireAfterSeconds:3600})
// Multi-document transaction
const session = client.startSession();
session.withTransaction(async () => {
await orders.insertOne({...}, {session});
await inventory.updateOne({_id:pid},{$inc:{qty:-1}},{session});
});
```

3.3 Sharding & Replication

• **Replica Set**: Primary (reads+writes) + Secondaries (replicate async) + optional Arbiter. Auto-failover.

• **Sharding**: Horizontal partitioning by shard key. Mongos router distributes queries. Config servers store metadata.

• **Shard key selection**: High cardinality, good write distribution, often queried in WHERE. Bad: monotonic timestamp (hot shard), low cardinality fields.

• **Read preference**: PRIMARY (default), PRIMARY_PREFERRED, SECONDARY (stale possible), NEAREST (lowest latency).

4. Apache Cassandra Deep Dive

4.1 Architecture & Concepts

• **Masterless**: every node is equal. No SPOF. Consistent hashing ring distributes data.

• **Partition key**: determines which node stores the data. All data for same partition key on same node(s).

• **Clustering columns**: sort order within partition. Define how rows within a partition are ordered.

• **Replication factor (RF)**: how many nodes hold copies. RF=3 means 3 copies.

• **Consistency level**: how many replicas must respond. QUORUM = majority. ONE = fastest.

```
-- Cassandra table: IoT sensor readings
CREATE TABLE sensor_data (
device_id UUID, -- partition key
recorded_at TIMESTAMP, -- clustering column (sort DESC)
temperature FLOAT,
humidity FLOAT,
PRIMARY KEY (device_id, recorded_at)
) WITH CLUSTERING ORDER BY (recorded_at DESC);
-- Query: last 100 readings for device (efficient — same partition)
SELECT * FROM sensor_data WHERE device_id=? LIMIT 100;
```

4.2 Consistency Levels

Level	Writes	Reads	Durability	Use Case
ONE	1 replica acks	1 replica responds	Low	Logging, metrics — speed over consistency
QUORUM	Majority acks	Majority responds	Medium	Most OLTP — good balance (RF=3: 2 nodes)
LOCAL_QUORUM	Majority in local DC	Majority in local DC	Medium	Multi-DC — avoid cross-DC latency
ALL	All replicas ack	All replicas respond	Highest	Critical writes — tolerate node down: no
ANY	At least 1 (can be hinted)	N/A	Lowest	Write-only, never query back

✔ Strong consistency: write QUORUM + read QUORUM (RF=3, W+R > RF). Tunable per operation.

■ Cassandra: design tables for query patterns. No joins. No aggregations across partitions. Model data how you query it.

5. Redis Deep Dive

5.1 Data Structures & Use Cases

Type	Commands	Realistic Use Case
String	SET, GET, INCR, DECR, APPEND, SETEX	Session tokens, counters, feature flags, distributed locks (SET NX EX).
Hash	HSET, HGET, HMSET, HGETALL, HDEL	User profile object, product details — field-level reads/writes.
List	LPUSH, RPUSH, LPOP, RPOP, LRANGE, BLPOP	Task queue (LPUSH jobs, BRPOP worker), activity feed, message history.
Set	SADD, SMEMBERS, SISMEMBER, SUNION, SINTER	Unique visitors per day, tags, follower list, deduplication.
Sorted Set	ZADD, ZRANGE, ZRANK, ZREVRANK, ZRANGEBYSCORE	Leaderboards, rate limiting (sliding window), delayed job scheduling.
Stream	XADD, XREAD, XGROUP, XACK	Event streaming, activity log, message queue with consumer groups.

HyperLogLog	PFADD, PFCOUNT	Approximate unique count (DAU, unique URLs) — fixed 12KB memory.
Bitmap	SETBIT, GETBIT, BITCOUNT	Daily active user tracking (bit per user per day), feature flags per user.

5.2 Redis Persistence, Clustering & Patterns

- **RDB (snapshot)**: point-in-time dump to disk (BGSAVE). Fast restart. Some data loss possible.
- **AOF (Append Only File)**: log every write command. More durable. Larger files. Rewrite periodically.
- **Redis Sentinel**: high availability — monitors master, auto-failover to replica.
- **Redis Cluster**: sharding across 16384 hash slots. Automatic slot distribution. Each slot on one master.
- **Distributed Lock** (Redlock): SET key value NX EX ttl — atomic acquire. Release only if value matches.

```
-- Distributed lock (SETNX + EXPIRE atomically)
SET lock:payment:order_123 client_uuid NX EX 30
-- NX = only set if not exists (acquire lock)
-- EX 30 = auto-expire in 30s (prevent deadlock)
```

6. DynamoDB Deep Dive

- **Fully managed** AWS KV + Document store. Single-digit millisecond latency at any scale.
- **Primary key**: Partition key only (simple), or Partition key + Sort key (composite). Design is critical.
- **GSI (Global Secondary Index)**: different partition+sort key — query by non-primary attributes.
- **LSI (Local Secondary Index)**: same partition key, different sort key — must be defined at creation.
- **Capacity modes**: Provisioned (RCU/WCU — predictable), On-Demand (pay per request — spiky).
- **DynamoDB Streams**: ordered log of item changes. Feed Lambda for event-driven processing.

```
// Single-table design: users + orders in one table
PK=USER#user_id SK=PROFILE --> user profile
PK=USER#user_id SK=ORDER#order_id --> user's order
GSI: PK=ORDER#order_id SK=STATUS --> query orders by status
```

✔ DynamoDB single-table design: store all entity types in one table, different SK patterns per entity. Enables hierarchical queries in one request.

7. SQL vs NoSQL Decision Guide

Scenario	Recommended	Why
User bank account, money transfer	PostgreSQL / MySQL	ACID transactions, consistency critical, relational data
Product catalog with variable attributes	MongoDB	Flexible schema — different products have different fields
Session storage for millions of users	Redis	O(1) KV lookup, TTL auto-expire, in-memory speed
IoT — billions of sensor readings/day	Cassandra / InfluxDB	High write throughput, time-series queries, linear scale
Social graph — 6 degrees of separation	Neo4j	Efficient relationship traversal — MATCH (u)-[:FRIEND*..6]->(v)
Full-text product search with filters	Elasticsearch	Inverted index, facets, relevance scoring, aggregations
Real-time leaderboard	Redis Sorted Set	ZADD O(log N), ZREVRANGE instant top-N, atomic increments

E-commerce cart for AWS-native app	DynamoDB	Single-digit ms at scale, flexible schema, streams for events
Analytics — group by, aggregations	PostgreSQL / Redshift	SQL power, window functions, columnar storage for OLAP
Multi-region globally distributed app	Cassandra / DynamoDB	Tunable consistency, multi-DC replication, no global lock

8. Interview Questions

SQL vs NoSQL

Q1. When would you choose NoSQL over SQL?

→ High scale (millions of writes/sec), flexible schema (attributes vary per entity), specific access patterns (always query by X), unstructured data, global distribution with tunable consistency. Not a blanket choice — relational data with complex queries still belongs in SQL.

Q2. What is eventual consistency? When is it acceptable?

→ System guarantees all replicas converge to same value given no new writes. Temporarily different values visible from different nodes. Acceptable: social feed (showing slightly old posts is fine), recommendation engine, DNS. NOT acceptable: bank balance, inventory count, seat booking.

Q3. What is CAP theorem and how do different NoSQL DBs fit?

→ Only 2 of 3 guarantees possible during network partition: Consistency (all nodes same data), Availability (every request responds), Partition tolerance (works despite network split). Cassandra: AP — available, eventually consistent. HBase: CP — consistent, may reject. MongoDB: CP (primary) or AP (secondary reads).

MongoDB

Q4. Embed vs reference in MongoDB — how do you decide?

→ Embed: data always accessed together, 1:1 or bounded 1:few, child doesn't exceed 16MB limit. Example: user embeds addresses. Reference: independently accessed, 1:many unbounded, shared across documents. Example: orders reference products. Start with embedding, normalize only when it causes problems.

Q5. What is a MongoDB replica set?

→ Group of mongod instances maintaining same data. Primary accepts writes, secondaries replicate async. If primary fails, secondaries elect a new primary (Raft-based). Write concern w:majority ensures data on majority before ack. Provides HA and read scaling.

Cassandra & Redis

Q6. Why can't you do arbitrary queries in Cassandra?

→ Cassandra stores rows sorted by partition key then clustering columns on disk. Efficient queries must match partition key (routes to right node) and can filter clustering columns in order. Ad-hoc queries require full cluster scan (ALLOW FILTERING) — anti-pattern. Model tables for known query patterns.

Q7. What is a hot partition in Cassandra or DynamoDB?

→ When too many requests hit the same partition key — one node overwhelmed while others idle. Cause: low cardinality shard key (e.g., status='active' for millions of orders) or monotonic keys (timestamps — all writes to latest partition). Fix: compound keys, add random suffix, use composite partition keys.

Q8. Redis eviction policies?

→ When memory full: noeviction (return error), allkeys-lru (evict least recently used), volatile-lru (LRU among keys with TTL), allkeys-random, volatile-random, volatile-ttl (evict soonest-expiring). For cache: allkeys-lru. For mixed (cache + persistent): volatile-lru.

Advanced

Q9. DynamoDB single-table design — why and how?

→ Single table avoids multiple round trips for related entities. Pattern: PK=entity type prefix + ID, SK=entity type or sub-key. GSI inverts PK/SK for reverse lookups. Example: PK=USER#123 SK=ORDER#456 for user's order. SK=USER#123 GSI allows orders-by-user and user-by-order queries from one table.

Q10. How do you handle schema migrations in MongoDB?

→ No built-in schema — options: (1) Lazy migration: check doc version on read, migrate in application. (2) Background migration script: update all docs in batches. (3) Dual-write: write both old and new format during transition. (4) Mongoose schema validation for soft enforcement.